

Task 1

Develop a C program that uses the system call `open()`, `read()`, `write()`. And `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source Code:

```
#include<stdio.h>

#include<fcntl.h>

#include<unistd.h>

#include<stdlib.h>

Int main(){

Int source, destination;

Char buffer[1024];

size_t bytesRead;

source=open("source.txt",O_RDONLY);

if(source < 0) {

    perror("Error opening source file");

    return 1;

}

destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

if(destination<0){

perror{"Error opening destination file"};

close(source);

return 1;

}

While((bytesRead=read(source,buffer,sizeof(buffer)))>0{

Write(destination,buffer,bytesReead);

}

Close(source);

Close(destination);
```

```
Printf("File copied successfully.\n");  
return 0;  
}
```

Output:

```
root@Tejasri:~# nano week2.c  
root@Tejasri:~# gcc week2.c -o week2  
root@Tejasri:~# ./week2  
File copied successfully.  
root@Tejasri:~# |
```

The program starts execution in user space.

The open() system call transfers control from user space to kernel space to open the file.

After opening the file, control returns to user space with a file descriptor.

The read() system call switches to kernel space to read data from the source file.

The kernel copies the data to the user buffer and returns control to the user space.

The write() system call switches to kernel space to write data to the destination file.

After writing the data, control returns to user space.

The close() system call switches to kernel space to close the files and release resources.

The kernel completes the operation and returns to user space, where the program terminates.

Task 2

use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequences or system calls and identify the kernel services involved

System Call	Purpose	Kernel Service
execve()	Executes the cat program	Process Management
openat()	Opens sample.txt	File System Management
read()	Reads data from the file	File I/O
write()	Displays data on the terminal	Output Management
close()	Closes the file	Resource Management
exit_group()	Terminates the process	Process Termination

Source code(sample.txt):

Hello Linux

Output:

```
root@Tejasri:~# echo "Hello Linux" > sample.txt
root@Tejasri:~# nano sample.txt
root@Tejasri:~# cat sample.txt
Hello Linux
root@Tejasri:~# strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffe01d18c08 /*
25 vars */) = 0
brk(NULL)                               = 0x619b4a2a0000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS
, -1, 0) = 0x72aaa2856000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such fil
e or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=19087, ...}) = 0
mmap(NULL, 19087, PROT_READ, MAP_PRIVATE, 3, 0) = 0x72aaa2851000
close(3)                                = 0
```